

Power Bi Introduction to Dax





What is DAX?

The Power Behind Power BI's Insights

“

DAX stands for Data Analysis Expressions. It is a formula language used to create Measures, Calculated Columns, and Calculated Tables in Power BI (as well as Power Pivot in Excel and SSAS Tabular Models).

Purpose: To perform powerful data calculations and create new information from your existing data model.

Think of it as: Excel formulas on steroids—it's similar to Excel but is designed to work with relational data tables and includes advanced functions for context, filtering, and time-based calculations.

”

```
=SUMX (
    FILTER (
        VALUES ( 'Date'[Year] ),
        'Date'[Year] < 2005
    ),
    IF (
        'Date'[Year] >= 2000,
        [Sales Amount] * 100,
        [Sales Amount] * 90
    )
)
```



DAX Formatting: Clarity is King

“

In DAX, formatting is everything. Not only does it make your code more readable, but it also helps clarify function nesting and execution context.

A simple rule to follow: If it is not formatted, it is not DAX you want to share or maintain.

- Well-Formatted vs. Badly Formatted DAX

The examples below demonstrate the difference. The well-formatted style is the recommended approach for any complex measure, as it makes the flow of the formula clear at a glance.

Badly Formatted DAX

This style is a single block of text that is difficult to read and debug. It hides the structure of the nested functions.

”

```
=CALCULATE(SUM(Sales[Amount]),  
FILTER(Sales,Sales[Amount]>500  
      &&
```

```
      Sales[Year]=2022))  
Total Sales 2022 High Value =  
    CALCULATE (  
        SUM ( Sales[Amount] ),  
        FILTER (  
            Sales,  
            Sales[Amount] > 500 &&  
            Sales[Year] = 2022  
        )  
    )
```



Calculated Columns in DAX

“

Calculated Columns are new columns added to your data model, defined using DAX formulas. They are unique because they are calculated row by row during data refresh and the resulting values are then stored in the model. The DAX expression for a calculated column, like Product[Price], implicitly refers to the current row being processed, meaning the final value is different for every row based on the data in that row.

Task1:

Provide the DAX formulas to create Calculated Columns for the customer's Age, the line-item Sales Amount, the Profit per line, the Extended Price (pre-discount), and the Shipping Days.

”

```
Age = IF(ISBLANK(Customer[Birth Date]), BLANK(), YEAR(TODAY()) - YEAR('Customer'[Birth Date]))
```

```
Sales Amount =  
Sales[Quantity] * (Sales[Unit Price] - Sales[Unit Cost])
```

```
Profit = (Sales[Unit Price] - 'Sales'[Unit Cost]) * 'Sales'[Quantity]
```

```
Extended Price = Sales[Unit Price] * Sales[Quantity]
```

```
Discount Amount =  
Sales[Extended Price] * Sales[Unit Discount]
```



Calculated Measures in DAX

“

Measures in DAX are dynamic calculations that are central to powerful reporting, unlike calculated columns which are static, row-level values. Measures don't calculate data row by row, but rather based on the Filter Context of a visual—meaning they adapt dynamically to any slicer, filter, or dimension applied in a report. Because measures can draw data from multiple tables and their definitions are stored separately from the physical data, the best practice for naming them is to exclude the table name (e.g use [Total Sales] instead of Sales [Total Sales]). This convention ensures flexibility, allowing measures to be easily moved to a dedicated measure table, and provides clarity by instantly identifying the item as a measure within your data model.

”

```
Total Sales =  
SUM('YourTable'[Net  
Price])
```

```
Total Quantity =  
SUM('YourTable'[Quantity])
```

```
Sales by Store =  
CALCULATE(  
SUM('YourTable'[Net  
Price]),  
ALLEXCEPT('YourTable',  
'YourTable'[StoreKey])  
)
```



Calculated Columns VS Measures

Use a Measure When...

You need to calculate Percentages or Ratios.

You need to perform Time Intelligence (YTD, MoM).

The calculation needs to be dynamic and respond to user filtering.

You need to conserve memory (Measures consume CPU).

Use a Calculated Column When...

You need to Slice or Filter a visual based on the result.

You need a Category, Attribute, or Label for each row.

The value must be static and stored in the data model.

You need a value visible in the Data View for inspection.



The DAX Functions Map: Your Analytical Toolkit

Now that we know the difference between **Calculated Columns** and **Measures**, it's time to explore the complete DAX language. This diagram shows the 9 **function categories** that make up DAX. We will cover the most important categories, like **Aggregation**, **Logical**, and especially **Filter** functions, to build the powerful, dynamic measures needed for professional reporting.





DAX Aggregation Functions: Summarizing Your Data

“

Aggregation functions in DAX are used to summarize numeric data across a column. Basic functions like SUM, AVERAGE, MIN, and MAX are straightforward, working directly on a single column of numbers to return a single result. However, when complex, row-level calculations are required before the final summary (such as calculating profit for each transaction), we must use the 'X' Aggregation Functions (Iterators) like SUMX, AVERAGEX, etc. These powerful functions iterate row by row over a specified table, evaluate a custom formula for each row, and then aggregate the results of that formula, ensuring accurate analysis for more complex business logic.

”

FUNCTIONX(Table,
Expression)

Task:

Total Revenue =
SUMX('Sales',
'Sales'[Quantity] *
'Sales'[Net Price])

Total Revenue =
SUMX('Sales',
'Sales'[Quantity] *
'Sales'[Net Price])



Summary of DAX Aggregation Functions

Function Category	Function	General Syntax
Standard Aggregation (Single Column)	COUNTROWS	COUNTROWS(Table)
Standard Aggregation (Single Column)	DISTINCTCOUNT	DISTINCTCOUNT(Column)
Standard Aggregation (Single Column)	SUM	SUM(Column)
Standard Aggregation (Single Column)	AVERAGE	AVERAGE(Column)
Standard Aggregation (Single Column)	MIN	MIN(Column)
Standard Aggregation (Single Column)	MAX	MAX(Column)
Iterator Aggregation (Row-by-Row Expression)	SUMX	SUMX(Table, Expression)
Iterator Aggregation	AVERAGEX	AVERAGEX(Table, Expression)
Iterator Aggregation	MAXX	MAXX(Table, Expression)
Iterator Aggregation	MINX	MINX(Table, Expression)



Using Variables (VAR)

“

Variables use a mandatory two-part structure: define one or more variables using the (VAR) keyword, each followed by an expression. The entire block must then conclude with the RETURN keyword, which specifies the single, final expression for the measure's output.

```
VAR <VariableName> = <Expression>  
RETURN <Final Expression>
```

”

```
# syntax : VAR <VariableName> =  
            <Expression>  
RETURN <Final Expression>  
Task:  
Gross Margin % =  
VAR TotalRevenue =  
    SUMX(  
        'Sales',  
        'Sales'[Quantity] * 'Sales'[Unit Price]  
    )  
  
VAR TotalCost =  
    SUMX(  
        'Sales',  
        'Sales'[Quantity] * 'Sales'[Unit Cost]  
    )  
  
VAR TotalMargin = TotalRevenue - TotalCost  
  
RETURN  
    DIVIDE(TotalMargin, TotalRevenue)
```

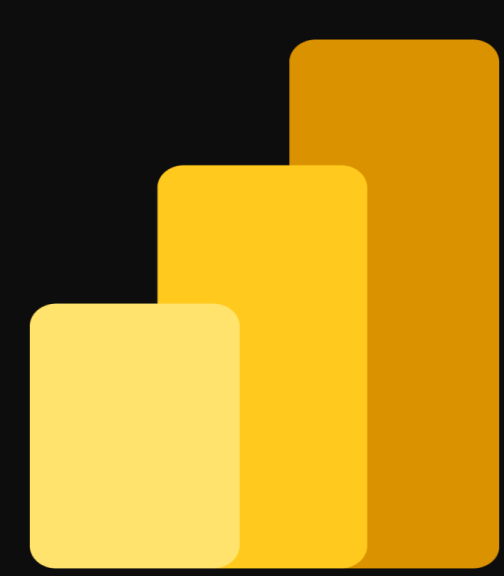


Table Functions: Manipulating Data Context

“

Table Functions are a crucial category of DAX that return an entire table as their result, rather than a single value. They are rarely used alone; instead, they serve as building blocks whose output (a modified table) is typically passed as an argument to other functions like (CALCULATE) or SUMX. Essential functions include FILTER (which returns a table meeting a condition), ALL (which removes filters from a table/column), and VALUES (which returns unique values in a column). Using these functions enables the complex logic needed to dynamically build custom filters and manage context transitions in advanced DAX calculations.

”



Summary of Table Functions and Syntax

Function	General Syntax	Used For
FILTER	FILTER}{Table}, Condition)	Creates a custom, filtered table.
ALL	ALL(table or column)	Removes filters to calculate a grand total.
VALUES	VALUES(Columns)	Returns a single-column table of unique values in the current context.
DISTINCT	DISTINCT(Column s)	Returns a single-column table of distinct values .
RELATEDTABLE	RELATEDTABLE(t able)	Returns a table of rows related to the current row.



The FILTER Function: Restricting Rows

“

The FILTER function is a crucial Table Function used to apply specific conditions to a table and restrict the number of rows it contains. Core Principles Returns a Table: FILTER does not return a single value; it always returns a new table that is a subset of the original. Input Required: It requires a table as its first argument, which can even be the result of another FILTER function (allowing nested filtering). Key Integration: It is most often combined with iterator functions (like SUMX or AVERAGEX) to run an aggregation over only the filtered rows.

”

Task:

```
High Quantity Sales =  
FILTER (  
    Sales,  
    Sales[Quantity] > 1  
)
```

```
High Cost Products =  
FILTER (  
    'Product',  
    'Product'[Unit Cost] >  
    10.00  
)
```



The ALL Function

“

The ALL function is a crucial Table Function in DAX that works by completely removing all existing filters from a specified table or column, always returning an entire table as its result.

Because ALL returns a table, it is not used alone but is frequently nested inside other functions like CALCULATE or SUMX. Its primary purpose is to ignore the active filter context of a visual or report, making it essential for accurately calculating totals, fixed ratios, or percentages of a grand total.

”

```
Unfiltered Product  
List =
```

```
ALL('Product')
```

```
All Available
```

```
Colors =
```

```
ALL('Product'[Color])
```



The DISTINCT Function: Unique Values

“

The DISTINCT function is a crucial Table Function in DAX that operates on a single column, returning a new table that contains only the unique values found in that column. Because it returns a table, it is often nested inside a function that requires a table input, such as COUNTROWS (to count the unique items), CALCULATE, or SUMX. The list of unique values returned is always filtered by the current context of the report (e.g., if you filter by a specific brand, DISTINCT only returns unique colors for that brand), making it essential for accurately counting unique entities like customers or products within a specific scope.

”

```
Unique Customers =  
COUNTROWS(  
DISTINCT('Sales'[CustomerKey])  
)
```

```
Distinct Colors Table =  
DISTINCT('Product'[Color])
```

The RELATABLE Function

“

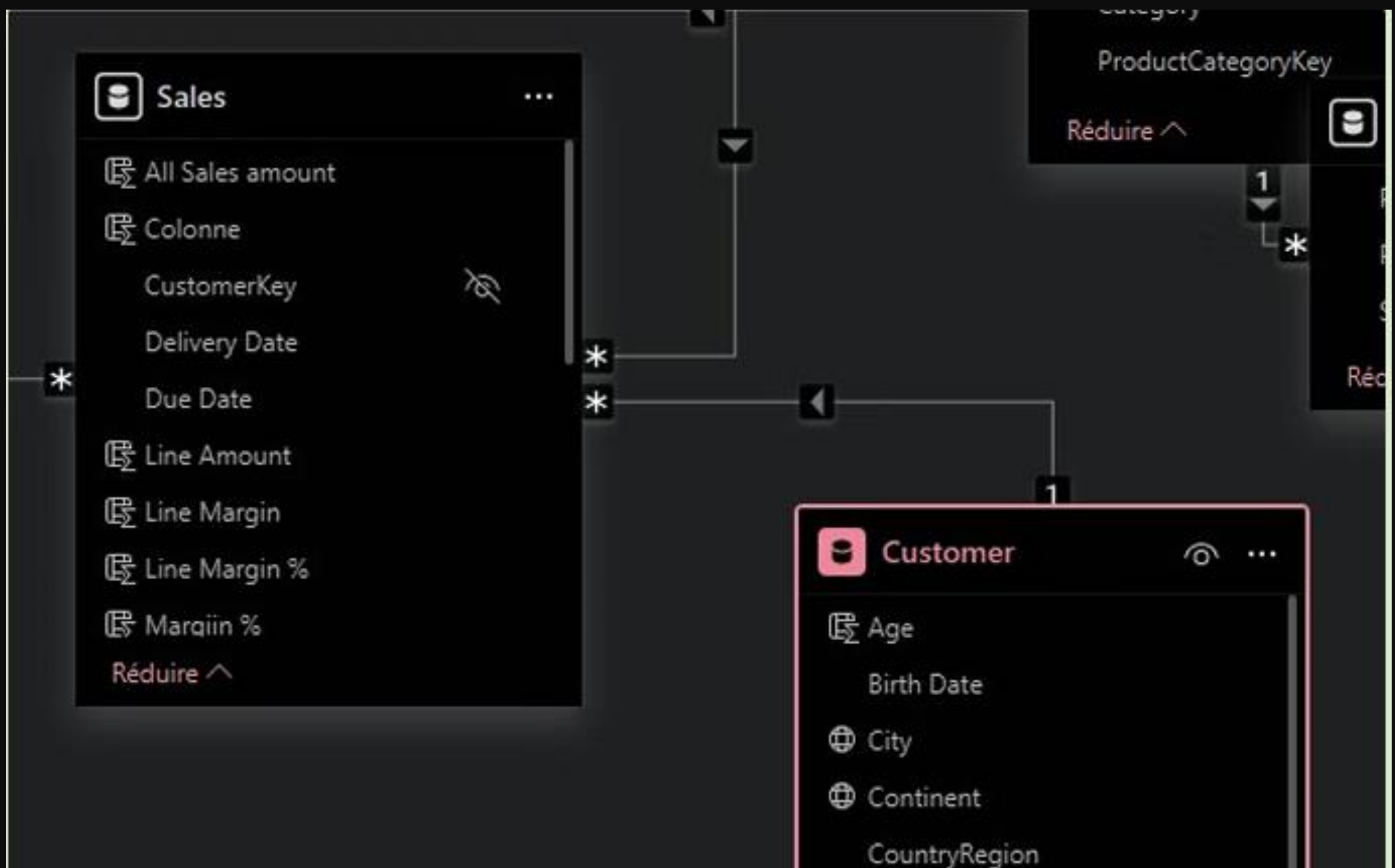
The RELATEDTABLE function is a Table Function used to return a table containing all the rows from another table that are related to the current row being evaluated. It must be used in a row context (like a calculated column or inside an iterator function like `SUMX`) and works by following the active one-to-many relationship defined in your data model. You don't need to explicitly specify the key columns when using RELATEDTABLE because DAX is a semantic model that automatically uses the data model relationships (the primary key / foreign key pair), operating at a higher level of abstraction than raw SQL where you would manually write JOIN ON syntax.

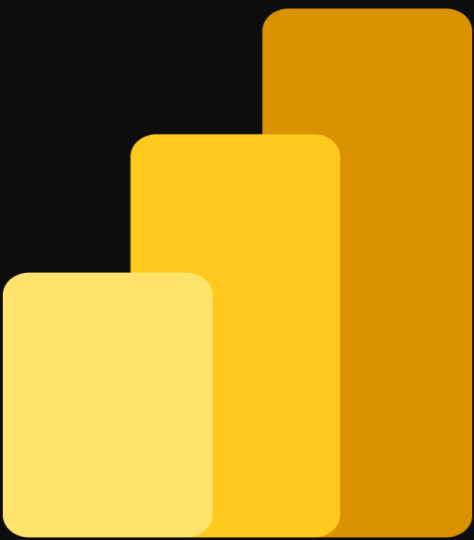
”

Task

```
Numbers of Sales =  
COUNTROWS(RELATEDTABLE(  
Sales))
```

```
Total Sales Per Product =  
SUMX(  
    RELATEDTABLE("Sales"), //  
    For the current Product, return  
    all related rows from the Sales  
    table  
    "Sales"[Quantity] *  
    "Sales"[Net Price] // Calculate  
    the revenue for each related  
    sale  
)
```





Evaluation Context in DAX

Evaluation Context is the "environment" or "set of rules" in which a DAX formula is calculated. It determines **which specific data points** in the model the formula is allowed to see and use.

The Two Main Types of Context

There are two primary contexts that govern every DAX calculation:

 Feature	 Row Context	 Filter Context
Focus	Focuses on a single row.	Focuses on a subset of data.
Math Style	Iterative (goes row by row).	Aggregative (sums/averages a group).
Typical Use	Calculated columns or SUMX.	Measures or Pivot Table cells.
Relationship	Does not follow relationships.	Follows relationships automatically.



Evaluation Context in DAX

- `CALCULATE(SUM(Sales[Amount]), Sales[Region] = "Europe")` // This creates a filter context
- where ONLY rows from Europe are used.

Why Context is Critical

Understanding Evaluation Context is the key to mastering DAX because formulas can behave completely differently depending on the context.

- **Avoid Errors:** It helps you avoid wrong totals and incorrect aggregations.
- **Advanced Logic:** It allows you to write advanced measures that use func

functions like `CALCULATE`, `FILTER`, and `ALL` properly to manipulate the flow of data.



Date Functions: Manipulating Dates and Time

“

Before utilizing the complex filter-shifting capabilities of Time Intelligence, it's essential to understand the basic Date Functions in DAX. These built-in functions allow you to work directly with temporal data by extracting specific parts of a date (like YEAR, MONTH, DAY, creating new date values (using DATE or DATEVALUE, or performing simple date arithmetic (like EDATE or EOMONTH.

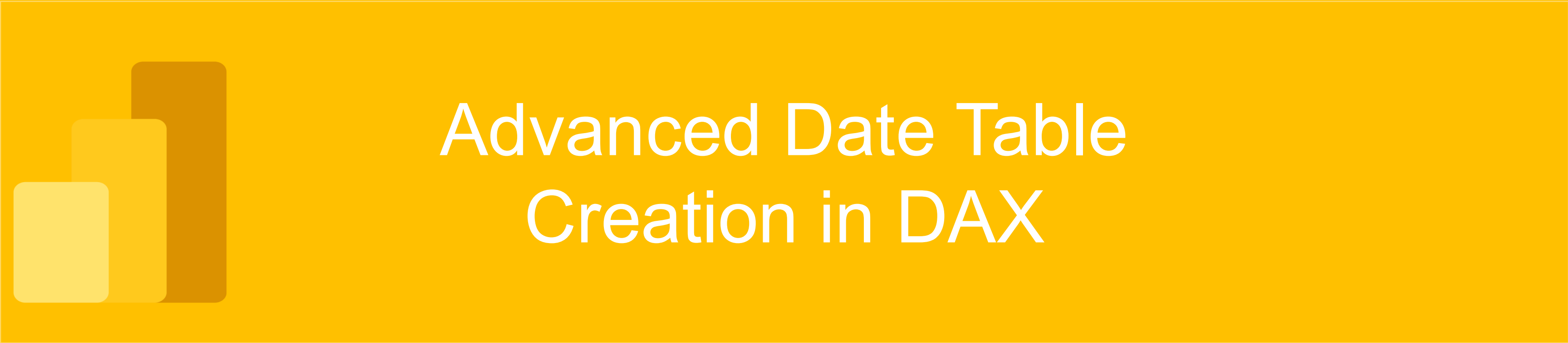
These functions are foundational, being used frequently to create the necessary attribute columns in your Date Table or to build simple, effective date-based filters and calculated columns across your model.

”



Date Functions: Built-in Date & Time Handling

Function Category	CategoryFunction	Purpose	Example
Extraction	YEAR, MONTH, DAY	Extracts the numerical year, month, or day from a date column.	MONTH('Sales'[Order Date])
	WEEKDAY, WEEKNUM	Returns the day of the week or the number of the week within the year.	WEEKDAY('Sales'[Order Date])
Creation	Combines separate year, month, and day inputs into a valid date.	Combines separate year, month, and day inputs into a valid date.	DATE(2025, 1, 1)
	DATEVALUE	Converts a text string representing a date into a proper Date/Time format.	DATEVALUE("10/30/2025")
Shifting	EDATE	Returns a date that is a specified number of months before or after the start date.	EDATE('Sales'[Order Date], 6)
	EOMONTH	Returns the last day of the month that is a specified number of months before or after the start date.	EOMONTH(TODAY(), 0)
Current Time	TODAY, NOW	TODAY() returns the current date; NOW() returns the current date and time.	TODAY()



Advanced Date Table Creation in DAX

A Date Table is **required** for most Time Intelligence calculations (like year-to-date or previous year comparisons). If your data model lacks one, you can create a robust version using a Calculated Table and DAX.

Rules for a Valid Date Table

- For DAX to use a table for Time Intelligence, it must adhere to these rules:
- **No Gaps:** All dates, from the first to the last day of the period, must be present (no missing dates or NULL).
 - **One Row Per Date:** Each date must occupy its own unique row.
 - **DateTime Column:** The table must contain at least one column of the DateTime data type.

The CALENDAR vs. CALENDARAUTO Trade-Off

DAX offers two basic table functions for date generation:

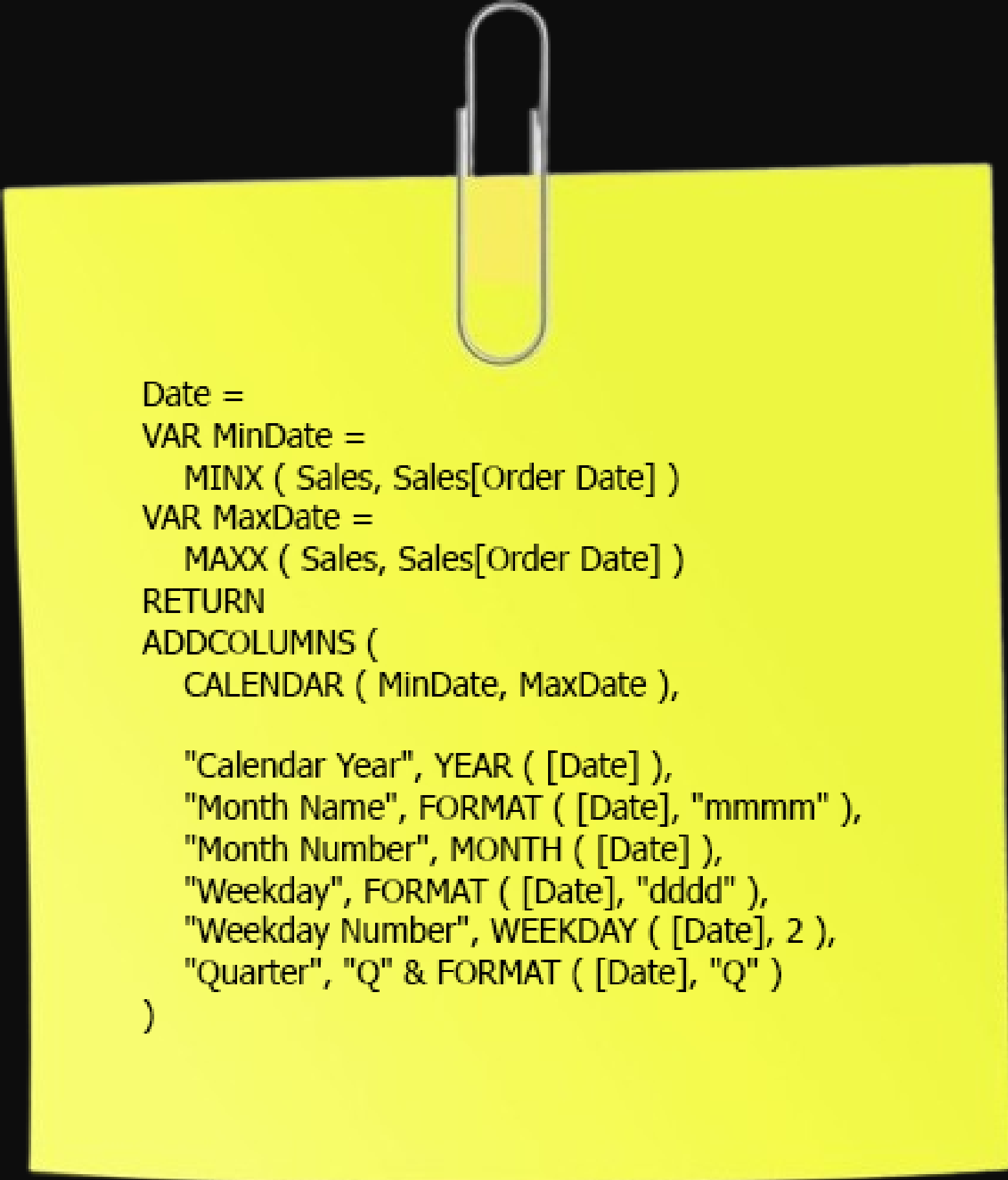
Function	How it Works	Limitation
CALENDARAUTO	Searches all date columns in the model and automatically finds the first and last year referenced.	Can include many irrelevant years if customer birth dates or other non-transaction dates are present.
CALENDAR	Requires explicit start and end dates.	Requires manually computing the minimum and maximum transaction dates, which is cumbersome.

Best Practice: Combining Functions for an Optimal Table

The most robust and reusable approach is to use a combination of VAR, MIN/MAX, CALENDARAUTO, and FILTERto ensure the Date Table only covers the range of your actual transactional data (like the '**Sales**' table).



Advanced Date Table Creation in DAX



```
Date =  
VAR MinDate =  
    MINX ( Sales, Sales[Order Date] )  
VAR MaxDate =  
    MAXX ( Sales, Sales[Order Date] )  
RETURN  
ADDCOLUMNS (   
    CALENDAR ( MinDate, MaxDate ),  
  
    "Calendar Year", YEAR ( [Date] ),  
    "Month Name", FORMAT ( [Date], "mmmm" ),  
    "Month Number", MONTH ( [Date] ),  
    "Weekday", FORMAT ( [Date], "dddd" ),  
    "Weekday Number", WEEKDAY ( [Date], 2 ),  
    "Quarter", "Q" & FORMAT ( [Date], "Q" )  
)
```

Once the **Date table** is in place, it is a good idea to **mark it as a Date table** in Power BI.

This ensures correct time intelligence calculations and relationships.



String Functions: Text Manipulation

DAX includes various functions designed to handle and manipulate text strings. These are often used in **Calculated Columns** for data cleanup or to create descriptive labels .

Function	Purpose	Example
CONCATENATE	Joins two text strings into one.	CONCATENATE('Product'[Brand], 'Product'[Color])
CONCATENATEX	CONCATENATEX: iterates over a table and joins all text results (useful for listing items).	CONCATENATEX(RELATEDTABLE('Sales'), 'Sales'[Product Name], ", ")
LEFT / RIGHT	Returns a specified number of characters from the start/end of a string.	LEFT('Product'[Product Name], 3)
MID	Returns a specific number of characters starting from a designated position.	MID('Product'[Product Name], 5, 4)
LEN	Returns the number of characters (length) in a text string.	LEN('Product'[Product Name])
UPPER / LOWER	Converts all characters in a string to uppercase or lowercase.	UPPER('Product'[Brand])
REPLACE	Replaces a specific number of characters in a string with a new string.	REPLACE('Product'[Code], 1, 3, "New")
SUBSTITUTE	Replaces all occurrences of a specific text within a string with new text.	SUBSTITUTE('Product'[Color], "r", "R")
TRIM	Removes extra space characters from the start and end of a text string.	TRIM('Product'[Name])



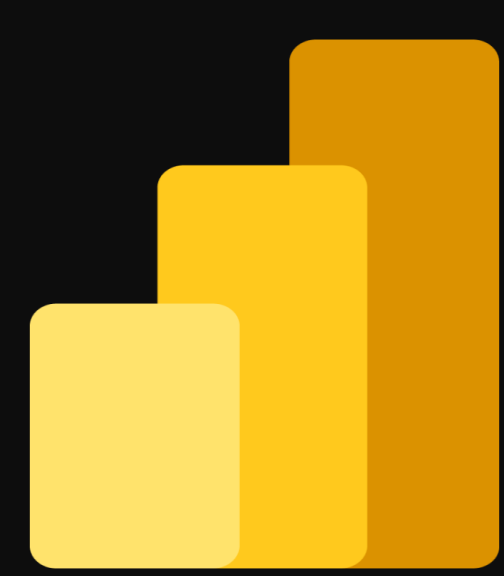
Logical Operations in DAX

Logical functions allow you to test conditions and return a value based on whether the result is TRUE or FALSE. They are foundational for creating sophisticated calculated columns and measures.

Function	Purpose	Example
IF	Checks whether a condition is met, and returns one value if TRUE, and another if FALSE.	IF('Sales'[Amount] > 100, "Large", "Small")
SWITCH	Evaluates an expression against a list of values and returns one of multiple results. Often cleaner than nested IF statements.	SWITCH(TRUE(), 'Sales'[Amount] > 1000, "High", 'Sales'[Amount] > 500, "Medium", "Low")
AND / OR	Checks if two or more logical expressions are both TRUE (AND) or if at least one is TRUE (OR).	AND('Product'[Color] = "Red", 'Product'[Brand] = "Contoso")
NOT	Reverses the logical value of its argument — converts TRUE → FALSE and FALSE → TRUE.	NOT(ISBLANK('Sales'[PromoCode]))
TRUE / FALSE	Explicitly returns a logical value TRUE or FALSE.	IF('Sales'[Quantity] > 0, TRUE(), FALSE())

Data Modeling





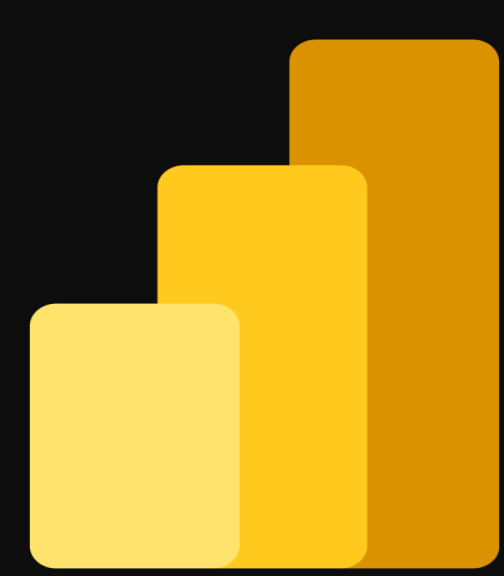
Defining Data Modeling

Definition:

- **Data modeling** is the process of **organizing data into logical tables** and **defining relationships** between them to support clear, consistent, and accurate analysis.
- It helps Power BI understand **how data from different sources connects**, ensuring that your visuals and calculations produce correct results.

Purpose of Data Modeling:

- To transform **raw data** into a **structured model** suitable for analysis.
- To define how tables **interact** and **filter** each other.
- To ensure reports remain **accurate**, **efficient**, and **scalable** as data grows.
- Just like a map helps you navigate a city, a data model helps Power BI navigate your data.



Key Concepts in Power BI Data Modeling

Fact Tables

- Contain **quantitative data** you want to analyze.
- Examples: Sales, transactions, revenue, order quantities.
- Usually have **numeric values** and foreign keys linking to dimension tables.

Dimension Tables

- Contain **descriptive or categorical data** that provides context for facts.
- Examples: Customers, Products, Regions, Time.
- Provide **filtering and grouping** capabilities in reports.



Key Concepts in Power BI Data Modeling

Definition:

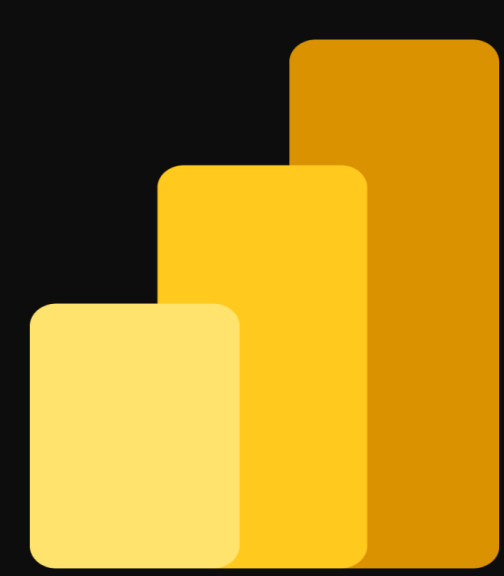
A relationship in Power BI connects two tables based on a common column, allowing data from multiple tables to interact in visuals and calculations.

Why Relationships Matter

- Ensures correct aggregation in reports.
- Helps create powerful insights without duplicating data.
- Allows you to build **fact tables** (transaction data) and **dimension tables** (reference data).
- Types of Relationships in Power BI

Cardinality defines the **number of unique records** that relate between two tables. It describes the **nature of the relationship**: how many rows in one table match rows in another.

Type	Description		Example
One-to-Many (1: *)	One record in Table A corresponds to multiple records in Table B.		Customers → Orders
Many-to-One (*:1)	Multiple records in Table A correspond to one record in Table B (inverse of above).		Orders → Customers
Many-to-Many (:)	Records in Table A correspond to multiple records in Table B and vice versa.		Products ↔ Suppliers
One-to-One (1:1)	Each record in Table A corresponds to exactly one record in Table B,		Each employee has exactly one email, and each email belongs to exactly one employee.



Key Concepts in Power BI Data Modeling

Schema:

The **structure of your data model** that defines how tables are organized and related.

- Determines **how data flows** between tables and **how calculations behave** in reports.

Star Schema:

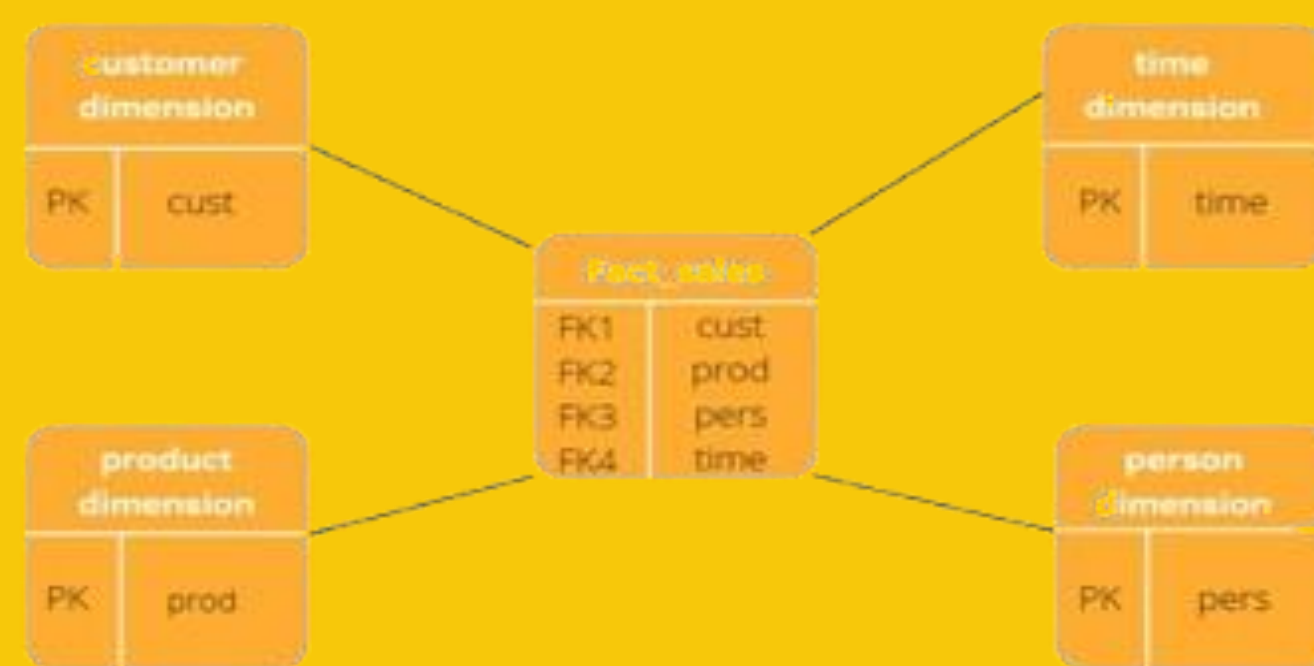
A **schema where a central fact table** connects directly to multiple **dimension tables**. Simple, fast for queries, and easy to understand.

Snowflake Schema:

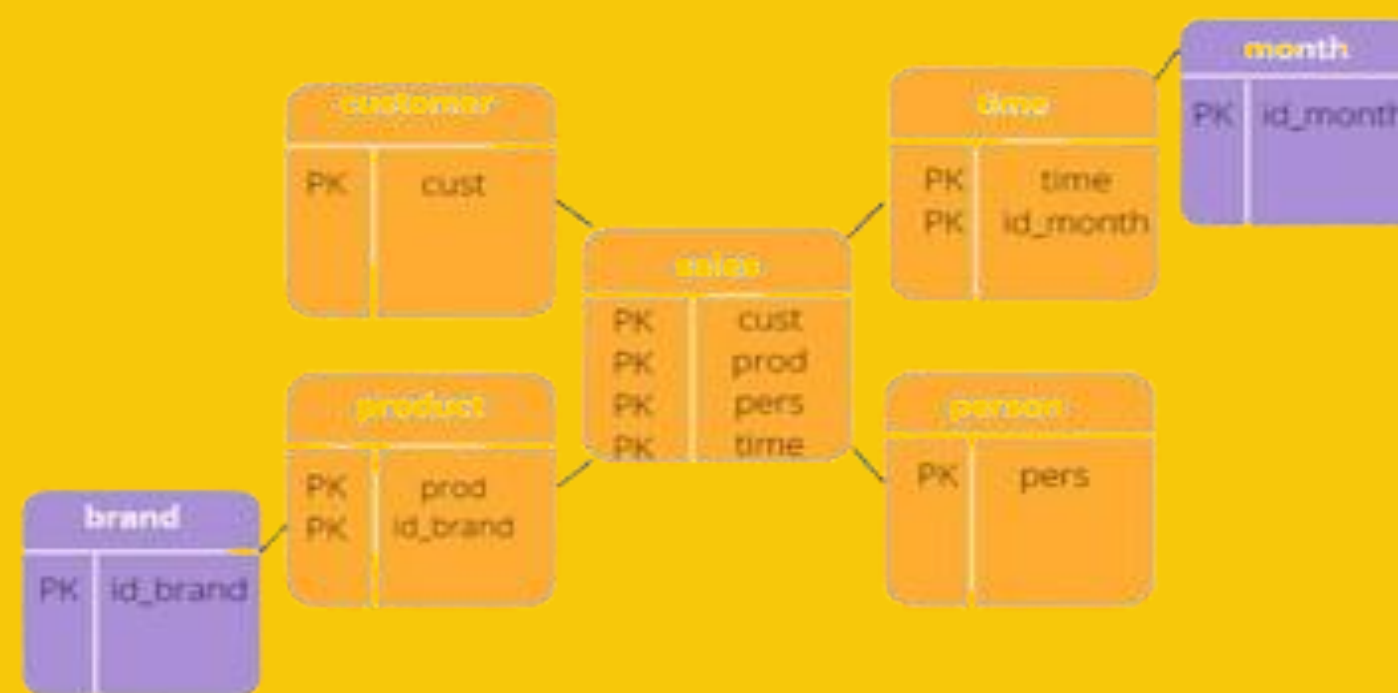
A **schema where dimension tables** are further normalized into related tables. More complex but saves storage and reduces data redundancy.

Star Schema Vs Snowflake schema

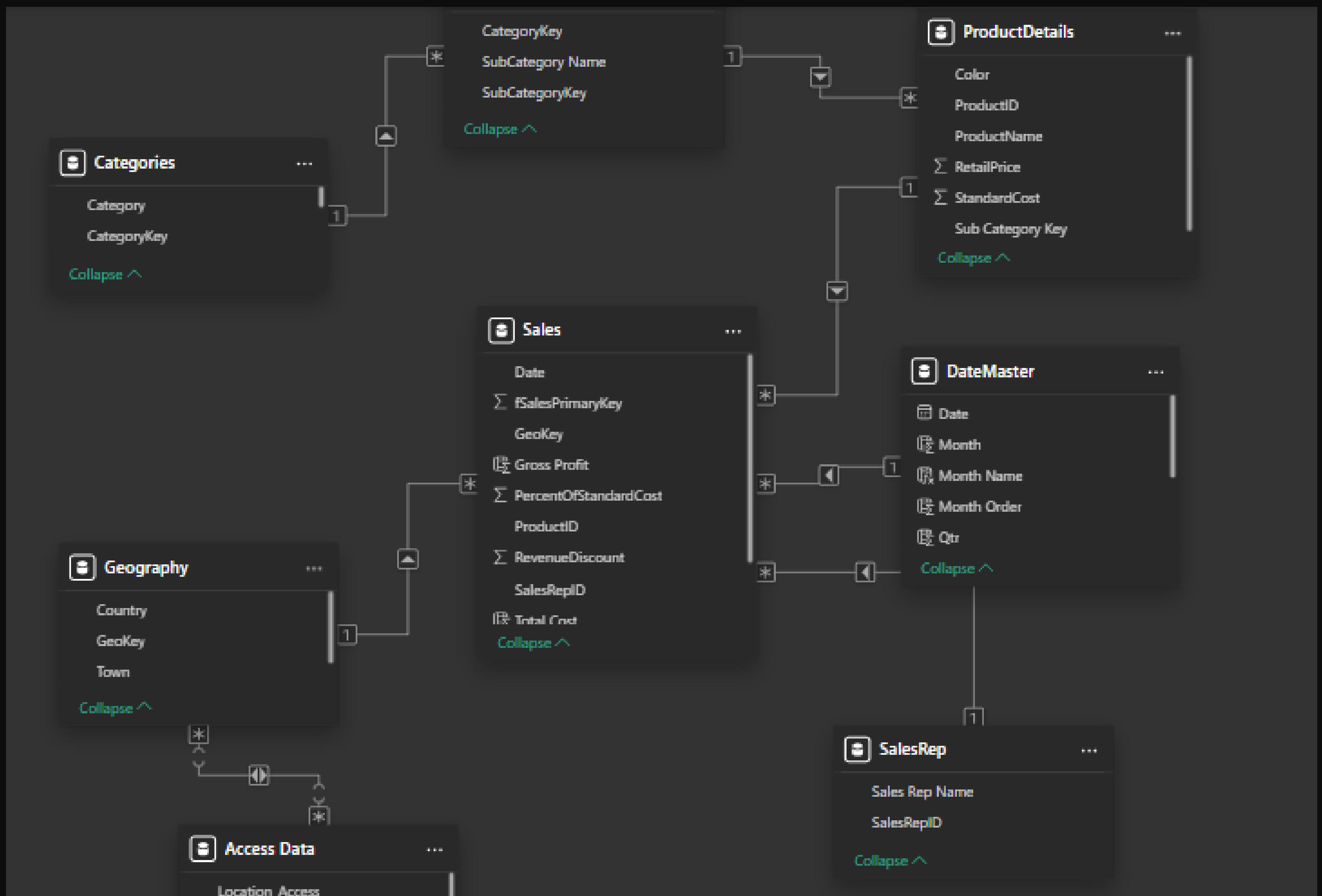
Star Schema



Snowflake Schema



The Power BI Data Model Example



This is the **data model** in Power BI — it visually represents how our data tables are organized and connected.

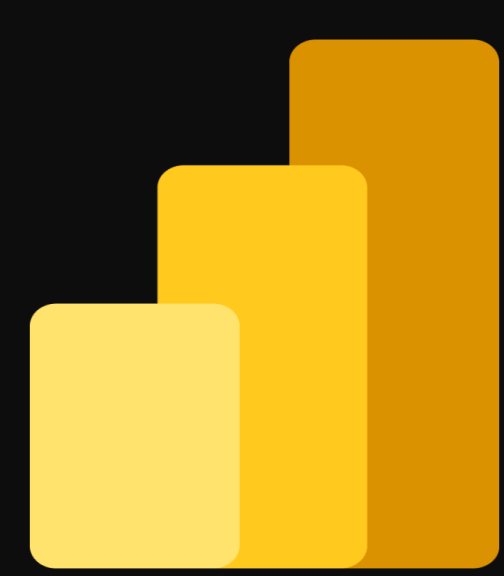
At the center, we have our *fact table* sales transactions, which stores the actual sales numbers.

Surrounding it are *dimension tables* describing products, customers, markets, and time.

Together, they form a **Star Schema**, which is ideal for analysis in Power BI.

Case Study: Retail Sales Insights Dashboard





Project Context and Objective

Imagine you've just joined a retail company as a **Data Analyst**. On your first day, your manager gives you your first assignment: "We need a **Sales Insights Dashboard** — something clear, automated, and interactive. It should help us track performance across products, regions, and sales reps. You'll have access to all the raw data — just make sense of it and make it easy for everyone to use." This is both exciting and challenging — a chance to build something from scratch that could directly support decision-making at the company.





Step 1: Understand the Requirements

- Define the goal: build a **one-page Power BI dashboard** to display sales insights across multiple dimensions — product, geography, and time.
- Identify and gather the required data sources:
 - Sales data (stored by year in a folder)
 - Categories (Excel file)
 - SubCategories (Excel file)
 - Sales Representatives (Excel file)
 - Product data (CSV or database)
 - Geography data (Excel file)
- Ensure the system design is **resilient**:
 - Removing a yearly sales file should not cause refresh errors.
 - Adding a new sales file should automatically include it in the dataset upon refresh.



Step 2: Data Preparation

•Task 11 - Automate Sales Data Loading

- Use **Power Query** to create a **dynamic loading mechanism** that imports all yearly sales files from the sales folder.
- Combine all files into a single **Sales Fact Table**.
- Ensure the model automatically updates when new files are added, without requiring manual intervention.



Step 3: Data Cleaning and Transformation

•Task 1.1 – Automate Sales Data Loading

•Task 2.1 – Split and Format Fields

- Split the **Location** field in the Sales table into two separate columns: *City* and *Country*.
- Assign appropriate **data types** for use in Power BI maps.
- Standardize the **Date field** to support time intelligence calculations in DAX.

•Task 2.2 – Create GeoKey

- Create a unique **GeoKey** column in both the **Sales** and **Geography** tables.
- Use this key to establish relationships between sales transactions and geographic locations.

•Task 2.3 – Clean ID Columns

- Identify columns in the **SalesRep** and **SubCategory** tables that contain IDs with the prefix “ID - ”.
- Build a reusable **Power Query function** that removes this prefix.
- Apply the function to both tables to clean and standardize the ID columns.

•Task 2.4 – Build the Data Model

- Design a **star schema** model:
 - Fact Table:** Sales
 - Dimension Tables:** Product, Category, SubCategory, SalesRep, Geography, Calendar
- Validate relationships between tables to ensure accurate aggregation and reporting.



Step 4: Create Business Measures with DAX

- Define DAX measures to calculate key performance metrics:
- **Total Revenue** = Units × Product Retail Price
- **Total Cost** = Units × Product Standard Cost
- **Gross Profit** = Total Revenue – Total Cost
- **Gross Profit MoM Growth %** = Month-over-Month change in Gross Profit
- **Average Sales per Day** = Total Revenue ÷ Number of Active Sales Days
- **QoQ Growth** = Quarter-over-Quarter growth (for QBR reporting)
- Verify all measures for accuracy and consistency using sample data.



Step 5: Design the Dashboard

- Use Power BI visuals to create an **interactive one-page dashboard**.
- Include the following elements:
 - **KPI Cards:** Total Revenue, Total Cost, Gross Profit
 - **Line Charts:** Month-over-Month (MoM) and Quarter-over-Quarter (QoQ) growth
 - **Maps:** Sales distribution by Country and City
 - **Bar Charts:** Top Products and Sales Representatives
 - **Filters/Slicers:** Year, Country, Category
- Ensure months are sorted chronologically from **January to December**.
- Design the layout for clarity, readability, and executive-level presentation.

Step 6: Deploy and Share the Report

- Publish the Power BI report following these deployment steps:
- Create a **Power BI Workspace** and assign appropriate user permissions.
- **Publish** the report to the Power BI Service.
- Configure a **Gateway** to enable on-premises data refresh.
- Set up **Scheduled Refreshes** for automated updates.
- Add **Roles, Subscriptions, and Alerts** for stakeholders.
- Package the report into a **Power BI App** and share it across departments.

Step 7: Evaluate Results

- **Verify that the dashboard:**

- Updates automatically when new yearly data is added.
- Provides accurate and real-time performance insights.
- Enables managers to analyze sales trends by product, region, and representative.
- Confirm that the dashboard supports **data-driven decisions** during quarterly business reviews (QBRs).
- Document feedback from management and optimize visuals or metrics if needed.